# END TO END LOG DATA ANALYTICS USING HDFS, KAFKA, MONGODB, PYSPARK AND STEAMLIT DOCUMENTATION

**Name:** **DEEPAK DEVAKUMAR S**

**R.No:** **RA2211047010076**

## Abstract:

In the era of large-scale data systems, real-time log analytics plays a crucial role in monitoring, diagnosing, and optimizing enterprise infrastructure. This project presents an end-to-end Big Data Log Analytics Pipeline built using Hadoop, Spark Structured Streaming, Apache Kafka, and MongoDB, with interactive dashboards powered by Streamlit. The primary goal is to process continuous log data from web servers, detect error trends (HTTP 5xx), and visualize system health metrics in real time. The architecture begins with Kafka, which ingests and streams live log data from multiple sources. The incoming data is processed through Spark Structured Streaming running on YARN, which performs parsing, tokenization, and aggregation operations over time-based windows. The processed results—such as hourly hit counts, error frequencies, and top URLs—are stored in MongoDB for persistent querying and alert generation. Visualization and alert modules are built using Streamlit, providing real-time dashboards for monitoring server performance, error spikes, and log trends. This pipeline demonstrates how distributed frameworks can be seamlessly integrated for real-time data ingestion, transformation, storage, and visualization. It ensures fault tolerance,

scalability, and automation through unified start/stop shell scripts. The project effectively showcases modern data engineering principles and the ability to operationalize real-time analytics on open-source big data technologies.

## PRE – REQUISITE: WSL2 UBUNTU

## 1. Project Overview

This project demonstrates a complete Big Data Log Analytics pipeline built on Ubuntu (WSL2) using:

1. Hadoop (HDFS + YARN) for distributed storage and resource management
2. Apache Spark Structured Streaming for real-time processing
3. Apache Kafka (KRaft) for message streaming
4. MongoDB for NoSQL storage and dashboard integration

The pipeline ingests Apache web server logs through Kafka, processes them in Spark (structured streaming), and stores aggregated results into MongoDB.

## 2. Environment Setup Summary

### Base System

1. OS: Ubuntu 22.04 (WSL2)

2. User: deepak
3. Java: OpenJDK 11
4. Python: 3.10 with virtual environment
5. Tools Directory: ~/tools
6. Project Directory: ~/spark-hdfs-mongo

## Commands used

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y openjdk-11-jdk python3-venv python3-pip build-essential git curl wget unzip
python3 -m venv ~/.venvs/bigdata
source ~/.venvs/bigdata/bin/activate
```

## Environment Variables

Added in ~/.bashrc:

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
export HADOOP_HOME=$HOME/tools/hadoop
export SPARK_HOME=$HOME/tools/spark
export KAFKA_HOME=$HOME/tools/kafka
export PATH=$HADOOP_HOME/bin:$HADOOP_HOME/sbin:$SPARK_HOME/bin:$KAFKA_HOME/bin:$PATH
export SPARK_LOCAL_IP=127.0.0.1
```

## 3. Hadoop Installation & Configuration

**Steps executed**

```
cd ~/tools
wget https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz -O hadoop.tgz
tar -xzf hadoop.tgz && mv hadoop-3.3.6 hadoop && rm hadoop.tgz
```

Edited core configs under $HADOOP_HOME/etc/hadoop/:

1. core-site.xml → defined fs.defaultFS = hdfs://localhost:9000
2. hdfs-site.xml → replication=1; name/data dirs set to /home/deepak/tools/hadoop_tmp
3. yarn-site.xml and mapred-site.xml → enabled YARN shuffle and MapReduce framework

**Formatting & Running:**

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
```

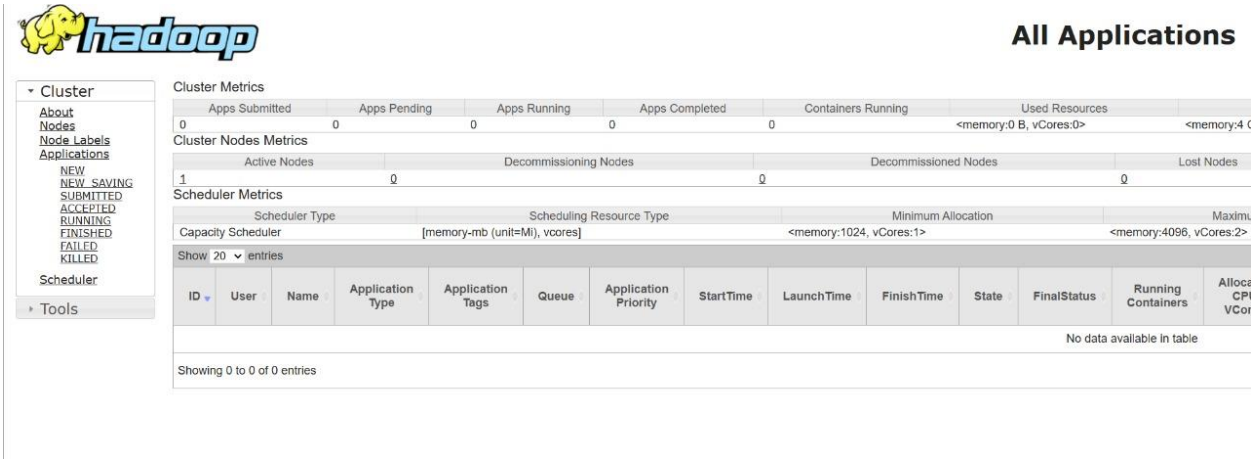
**Verification:**

```
hdfs dfs -mkdir -p /user/deepak
hdfs dfs -ls /
```

**Web UIs:**

1. HDFS → http://localhost:9870

2. YARN → http://localhost:8088




## 4. Spark Installation

```
cd ~/tools
wget https://archive.apache.org/dist/spark/spark-3.5.2/spark-3.5.2-bin-hadoop3.tgz -O spark.tgz
tar -xzf spark.tgz && mv spark-3.5.2-bin-hadoop3 spark && rm spark.tgz
```

## Test run:

```
spark-submit --master local[2] \
  --class org.apache.spark.examples.SparkPi \
  $SPARK_HOME/examples/jars/spark-examples_2.12-3.5.2.jar 100
```

## 5. Kafka (KRaft) Setup

```
cd ~/tools
wget https://archive.apache.org/dist/kafka/3.7.0/kafka_2.13-3.7.0.tgz -O kafka.tgz
tar -xzf kafka.tgz && mv kafka_2.13-3.7.0 kafka && rm kafka.tgz
```

Modified config/kraft/server.properties:

```
process.roles=broker,controller
node.id=1
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
controller.quorum.voters=1@localhost:9093
log.dirs=/home/deepak/tools/kafka/data
```

**Initialize cluster & start:**

```
export KAFKA_CLUSTER_ID=$(bin/kafka-storage.sh random-uuid)
bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties
bin/kafka-server-start.sh config/kraft/server.properties
```

**Topic creation:**

```
bin/kafka-topics.sh --create --topic web-logs --bootstrap-server localhost:9092
bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

## 6. MongoDB Installation

```
curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc | sudo
gpg --dearmor -o /usr/share/keyrings/mongodb-server-8.0.gpg
echo "deb [ arch=amd64 signed-by=/usr/share/keyrings/mongodb-
server-8.0.gpg ] https://repo.mongodb.org/apt/ubuntu
jammy/mongodb-org/8.0 multiverse" | sudo tee
/etc/apt/sources.list.d/mongodb-org-8.0.list
sudo apt update && sudo apt install -y mongodb-org
sudo systemctl start mongod && sudo systemctl enable mongod
```

### Verification:

```
mongosh --eval 'db.runCommand({ ping: 1 })'
```

## 7. Spark Streaming Job (Kafka → MongoDB)

### Project Structure

```
spark-hdfs-mongo/
    src/
     └── stream_kafka_to_mongo.py
    sample_logs/
     └── access.log
    scripts/
     └── produce_kafka.sh
```

## Producer Script

```
#!/bin/bash
TOPIC="web-logs"
FILE="~/spark-hdfs-mongo/sample_logs/access.log"
$KAFKA_HOME/bin/kafka-console-producer.sh --bootstrap-server
localhost:9092 --topic "$TOPIC" < "$FILE"
```

## 8. HDFS Storage :

### For checkpoints(Fault tolerance)

```
hdfs dfs -mkdir -p /data/logs
hdfs dfs -put ~/spark-hdfs-mongo/sample_logs/access.log /data/logs/
hdfs dfs -ls /data/logs
```

## 9) PROJECT SECTION:

## Section 1 – Project Setup Validation

### JPS + Directory listing (spark-hdfs-mongo root)

```
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ jps
23603 SecondaryNameNode
23317 DataNode
23847 ResourceManager
24443 Kafka
23166 NameNode
23999 NodeManager
26079 Jps
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ ls
app   notify_alerts.py  pipeline.sh       requirements.txt  run_all.sh   scripts  start_all.sh  stop_all.sh
logs  pids              produce_kafka.sh  restart_all.sh    sample_logs  src      status.sh
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$
```

**Description:**

This screenshot shows all major services up and running on the WSL2 Ubuntu environment — NameNode, DataNode, SecondaryNameNode,

ResourceManager, NodeManager, and Kafka. The presence of these Java daemons confirms successful startup of the Hadoop and Kafka clusters.

Commands:

```
jps        # verify all Java daemons

ls         # confirm project structure
```

## Section 2 — Real-Time Stream: Kafka → Spark Structured Streaming → MongoDB

### Purpose

In this component, Apache Spark consumes log lines from a Kafka topic in real time, performs a lightweight transformation (tokenization + counts), and upserts aggregated results into MongoDB. The stream is checkpointed to HDFS for fault tolerance and exactly-once semantics per micro-batch.

## Screenshot 1 – Spark Kafka Stream Initialization (`stream_kafka_to_mongo.py`)

```
GNU nano 7.2                                    stream_kafka_to_mongo.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, split, explode, current_timestamp
from pyspark.sql.types import StringType

# Create Spark session
spark = SparkSession.builder.appName("KafkaLogsToMongo").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

# === Read from Kafka (structured streaming) ===
df_raw = (
    spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", "localhost:9092")
    .option("subscribe", "logs")
    .option("startingOffsets", "latest")
    .load()
)

# Parse to text -> tokenize -> count
lines = df_raw.select(col("value").cast(StringType()).alias("line"))
tokens = (
    lines.select(explode(split(col("line"), r"\s+")).alias("token"))
    .where((col("token").isNotNull()) & (col("token") != ""))
)
counts = tokens.groupBy("token").count().withColumn("ingested_at", current_timestamp())
                                                                [ Read 62 lines ]
```

```
GNU nano 7.2                                    stream_kafka_to_mongo.py
# === Sir evidence ===
print("=== printSchema ===")
counts.printSchema()
print("\n=== explain(True) ===")
counts.explain(True)

# Show preview each micro-batch
def show_batch(df, epoch_id):
    print("\n=== preview (show 10) ===")
    df.orderBy(col("count").desc()).show(10, truncate=False)

# Write to MongoDB using PyMongo from the driver
from pymongo import MongoClient

def write_to_mongo(df, epoch_id):
    rows = [r.asDict() for r in df.collect()]
    if not rows:
        return
    client = MongoClient("mongodb://localhost:27017")
    col = client["bdproj"]["agg_results"]
    for r in rows:
        col.update_one(
            {"token": r["token"]},
            {"$inc": {"count": int(r["count"])}, "$set": {"last_ingested_at": r["ingested_at"]}},
```

## Description:

This image shows the creation of a Spark Structured Streaming session named “KafkaLogsToMongo”.

Spark connects to the Kafka broker on `localhost:4041` and subscribes to the topic logs.

configuration`.option("startingOffsets","latest")` ensures the stream processes only newly arriving messages.
This step establishes the real-time ingestion pipeline where Spark continuously receives log lines from Kafka.

## Screenshot 2 – Tokenization and Aggregation

```
  GNU nano 7.2                                          stream_logs.py
from pyspark.sql import SparkSession, functions as F

# same regex you used before (Apache/Nginx common log with quotes)
LOG_RE = r'^(\S+)\s+\S+\s+\S+\s+\[([^\]]+)\]\s+"(\S+)\s?([^"]*)\s?(\S+)?"\s+(\d{3})\s+(\S+)\s+"[^"]*"\s+"([^"]*)"'

MONGO_URI = "mongodb://127.0.0.1:27017"
DB = "bdproj"
ERROR_THRESHOLD = 1  # raise alert if >= this many 5xx in a (date,hour) batch

def parse(df):
    df = (df.select(F.col("value").alias("line"))
             .withColumn("ip",     F.regexp_extract("line", LOG_RE, 1))
             .withColumn("ts_str", F.regexp_extract("line", LOG_RE, 2))
             .withColumn("method", F.regexp_extract("line", LOG_RE, 3))
             .withColumn("url",    F.regexp_extract("line", LOG_RE, 4))
             .withColumn("status", F.regexp_extract("line", LOG_RE, 6).cast("int"))
             .withColumn("bytes",  F.when(F.regexp_extract("line", LOG_RE, 7) == "-", 0)
                                      .otherwise(F.regexp_extract("line", LOG_RE, 7).cast("long")))
             .withColumn("agent",  F.regexp_extract("line", LOG_RE, 8))
             .drop("line"))
    df = (df
          .withColumn("ts", F.to_timestamp("ts_str", "dd/MMM/yyyy:HH:mm:ss Z"))
          .withColumn("date", F.to_date("ts"))
          .withColumn("hour", F.hour("ts"))
```

## Description:

Here, Spark transforms raw Kafka messages into textual tokens using `split()` and `explode()`.
Each line is broken into individual words (“tokens”), empty values are filtered, and Spark groups them by token to compute frequency counts.
The column `ingested_at` adds the current timestamp for each micro-batch, providing a time marker for MongoDB updates.

## Screenshot 3 – Schema Evidence and MongoDB Writer

```
+---------+-------------------+------+----------------------+------+-----+-----------+
|127.0.0.1|2025-10-22 18:02:00|GET   |/api/v1/users HTTP/1.1 |200   |512  |Mozilla/5.0|
|127.0.0.1|2025-10-22 18:02:00|GET   |/api/v1/orders HTTP/1.1|200   |1024 |Mozilla/5.0|
|127.0.0.1|2025-10-22 18:02:00|POST  |/api/v1/orders HTTP/1.1|201   |256  |curl/8.0   |
|127.0.0.1|2025-10-22 18:02:00|GET   |/health HTTP/1.1       |200   |64   |curl/8.0   |
|127.0.0.1|2025-10-22 18:02:00|GET   |/api/v1/users HTTP/1.1 |500   |0    |Mozilla/5.0|
+---------+-------------------+------+----------------------+------+-----+-----------+
only showing top 5 rows

ALERTS:
+----------+----+----+------+--------+-------------------------+
|date      |hour|hits|errors|epoch_id|created_at               |
+----------+----+----+------+--------+-------------------------+
|2025-10-22|18  |18  |4     |1035    |2025-10-22 18:02:21.070107|
+----------+----+----+------+--------+-------------------------+

=== Batch 1036 sample ===
+---------+-------------------+------+----------------------+------+-----+-----------+
|ip       |ts                 |method|url                   |status|bytes|agent      |
+---------+-------------------+------+----------------------+------+-----+-----------+
|127.0.0.1|2025-10-22 18:02:15|GET   |/api/v1/users HTTP/1.1 |200   |512  |Mozilla/5.0|
|127.0.0.1|2025-10-22 18:02:15|GET   |/api/v1/orders HTTP/1.1|200   |1024 |Mozilla/5.0|
|127.0.0.1|2025-10-22 18:02:15|POST  |/api/v1/orders HTTP/1.1|201   |256  |curl/8.0   |
|127.0.0.1|2025-10-22 18:02:15|GET   |/health HTTP/1.1       |200   |64   |curl/8.0   |
|127.0.0.1|2025-10-22 18:02:15|GET   |/api/v1/users HTTP/1.1 |500   |0    |Mozilla/5.0|
+---------+-------------------+------+----------------------+------+-----+-----------+
only showing top 5 rows
```

```
bdproj> const today = new Date(); today.setHours(0,0,0,0);
... db.top_urls.find({ date: { $gte: today } })
...            .sort({ hits: -1 }).limit(10)
[
  {
    _id: ObjectId('68f91be518afa064bb0ab32b'),
    date: ISODate('2025-10-22T00:00:00.000Z'),
    url: '/api/v1/users HTTP/1.1',
    hits: Long('88')
  },
  {
    _id: ObjectId('68f91be518afa064bb0ab32c'),
    date: ISODate('2025-10-22T00:00:00.000Z'),
    url: '/api/v1/orders HTTP/1.1',
    hits: Long('66')
  },
  {
    _id: ObjectId('68f91be518afa064bb0ab32e'),
    date: ISODate('2025-10-22T00:00:00.000Z'),
    url: '/health HTTP/1.1',
    hits: Long('22')
  },
  {
    _id: ObjectId('68f91be518afa064bb0ab32f'),
    date: ISODate('2025-10-22T00:00:00.000Z'),
    url: '/api/v1/payments HTTP/1.1',
    hits: Long('22')
  },
  {
    _id: ObjectId('68f91c2718afa064bb0ab381'),
```

## Description:

This section prints the streaming DataFrame's schema and execution plan (printSchema() and explain(True)) to verify Spark's structure before execution.

The function write_to_mongo() uses **PyMongo** to upsert each token's

count into the MongoDB collection bdproj.agg_results.
It performs $inc operations to accumulate counts and $set to record the last_ingested_at timestamp, ensuring incremental and fault-tolerant updates.

## Screenshot 4 – Streaming Query Execution

```
GNU nano 7.2                                    stream_logs_kafka.py
def main():
    spark = (SparkSession.builder
             .appName("stream_log_analytics_kafka_to_mongo")
             .config("spark.mongodb.write.connection.uri", MONGO_URI)
             .getOrCreate())
    spark.sparkContext.setLogLevel("WARN")

    kafka = (spark.readStream
             .format("kafka")
             .option("kafka.bootstrap.servers", "localhost:9092")
             .option("subscribe", "web_logs")
             .option("startingOffsets", "latest")
             .load())

    lines = kafka.select(F.col("value").cast("string").alias("text"))
    parsed = parse_from_text_col(lines, "text")

    # --- EVIDENCE: schema + physical plan printed once at startup ---
    parsed.printSchema()
    parsed.explain(True)
    print("Schema and plan printed above. Waiting for micro-batches...")

    query = (parsed.writeStream
             .foreachBatch(process_batch)
```

```
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ hdfs dfs -ls /checkpoints
Found 1 items
drwxr-xr-x   - deepak supergroup          0 2025-10-15 22:21 /checkpoints/stream_logs_kafka
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$
```

## Description:

This final segment defines the active query using .writeStream.
The stream operates in update mode, invoking both show_batch() for preview and write_to_mongo() for database insertion on every micro-batch.
A checkpoint directory is defined in HDFS at /checkpoints/kafka_logs to store progress and allow recovery after restarts.
The call to .awaitTermination() keeps the job running indefinitely, awaiting continuous Kafka input.

## Screenshot 5 – Sample Logs and Log Replay Script (produce_kafka.sh)

```
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ cd sample_logs
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo/sample_logs$ ls
access.log
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo/sample_logs$ cat access.log
127.0.0.1 - - [09/Oct/2025:10:01:11 +0530] "GET /api/v1/users HTTP/1.1" 200 512 "-" "Mozilla/5.0"
127.0.0.1 - - [09/Oct/2025:10:02:30 +0530] "GET /api/v1/orders HTTP/1.1" 200 1024 "-" "Mozilla/5.0"
127.0.0.1 - - [09/Oct/2025:10:02:59 +0530] "POST /api/v1/orders HTTP/1.1" 201 256 "-" "curl/8.0"
127.0.0.1 - - [09/Oct/2025:10:03:12 +0530] "GET /health HTTP/1.1" 200 64 "-" "curl/8.0"
127.0.0.1 - - [09/Oct/2025:10:04:03 +0530] "GET /api/v1/users HTTP/1.1" 500 0 "-" "Mozilla/5.0"
127.0.0.1 - - [09/Oct/2025:11:00:05 +0530] "GET /api/v1/users HTTP/1.1" 200 768 "-" "Mozilla/5.0"
127.0.0.1 - - [09/Oct/2025:11:01:45 +0530] "GET /api/v1/orders HTTP/1.1" 404 0 "-" "Mozilla/5.0"
127.0.0.1 - - [09/Oct/2025:11:05:10 +0530] "GET /api/v1/users HTTP/1.1" 200 256 "-" "Mozilla/5.0"
127.0.0.1 - - [17/Oct/2025:23:28:55 +0530] "GET /api/v1/payments HTTP/1.1" 503 0 "-" "Mozilla/5.0"
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo/sample_logs$
```

```
  GNU nano 7.2                                   produce_kafka.sh
#!/bin/bash
set -euo pipefail
LOGFILE="$HOME/spark-hdfs-mongo/sample_logs/access.log"
TOPIC="web_logs"
KAFKA_BIN="$HOME/kafka/bin/kafka-console-producer.sh"
BROKER="localhost:9092"

echo "♻  Replaying $LOGFILE to $TOPIC with LIVE timestamps (Ctrl+C to stop)"
while true; do
  while IFS= read -r line; do
    now=$(date -u +"%d/%b/%Y:%H:%M:%S +0000")
    # replace the bracketed timestamp with NOW (UTC)
    echo "$line" | sed -E "s|\[[^]]+\]|[${now}]|"
  done < "$LOGFILE" | "$KAFKA_BIN" --bootstrap-server "$BROKER" --topic "$TOPIC"
  echo "→ Completed one cycle — restarting in 5s..."
  sleep 5
done
```

## Description:

This Bash script simulates live log streaming by reading lines from sample_logs/access.log and sending them to the Kafka topic using the built-in kafka-console-producer.sh.

It dynamically replaces log timestamps with the current UTC time, creating a continuous feed loop that replays logs every 5 seconds.

This ensures that Spark receives fresh log data in real time for testing and analytics.

## Screenshot 6 – Log Parsing and Transformation (stream_logs.py)

```
  GNU nano 7.2                                          stream_logs.py
from pyspark.sql import SparkSession, functions as F

# same regex you used before (Apache/Nginx common log with quotes)
LOG_RE = r'^(\S+)\s+\S+\s+\S+\s+\[([^\]]+)\]\s+"(\S+)\s?([^"]*)\s?(\S+)?"\s+(\d{3})\s+(\S+)\s+"[^"]*"\s+"([^"]*)"'

MONGO_URI = "mongodb://127.0.0.1:27017"
DB = "bdproj"
ERROR_THRESHOLD = 1  # raise alert if >= this many 5xx in a (date,hour) batch

def parse(df):
    df = (df.select(F.col("value").alias("line"))
             .withColumn("ip",     F.regexp_extract("line", LOG_RE, 1))
             .withColumn("ts_str", F.regexp_extract("line", LOG_RE, 2))
             .withColumn("method", F.regexp_extract("line", LOG_RE, 3))
             .withColumn("url",    F.regexp_extract("line", LOG_RE, 4))
             .withColumn("status", F.regexp_extract("line", LOG_RE, 6).cast("int"))
             .withColumn("bytes",  F.when(F.regexp_extract("line", LOG_RE, 7) == "-", 0)
                                     .otherwise(F.regexp_extract("line", LOG_RE, 7).cast("long")))
             .withColumn("agent",  F.regexp_extract("line", LOG_RE, 8))
             .drop("line"))
    df = (df
          .withColumn("ts", F.to_timestamp("ts_str", "dd/MMM/yyyy:HH:mm:ss Z"))
          .withColumn("date", F.to_date("ts"))
          .withColumn("hour", F.hour("ts"))
```

```
  GNU nano 7.2                                          stream_logs.py
def process_batch(batch_df, epoch_id):
    if batch_df.isEmpty():
        return

    # === Aggregations on this micro-batch ===
    hits_by_hour = (batch_df.groupBy("date","hour")
                    .agg(F.count("*").alias("hits"),
                         F.sum(F.when(F.col("status") >= 500, 1).otherwise(0)).alias("errors"))
                    .orderBy("date","hour"))

    top_urls = (batch_df.groupBy("date","url")
                .agg(F.count("*").alias("hits"))
                .orderBy(F.desc("hits"))
                .limit(1000))

    status_counts = (batch_df.groupBy("date","status")
                     .agg(F.count("*").alias("count"))
                     .orderBy("date","status"))

    # write each table to MongoDB (append streaming results)
    write_mongo(hits_by_hour, "hits_by_hour", mode="append")
    write_mongo(top_urls, "top_urls", mode="append")
    write_mongo(status_counts, "status_counts", mode="append")
```

## Description:

The script parses raw log text into structured fields (IP, timestamp, method, URL, status, bytes, user-agent) using a **regular expression** matching the Apache/Nginx Common Log Format.

Timestamps are converted to Spark timestamp, date, and hour columns for temporal aggregation.

This forms the foundation for detailed analytics such as traffic trends and error monitoring.

## Screenshot 7 – Aggregation Logic

```
GNU nano 7.2                                    analytics.py
if not df_status.empty:
    latest_day = df_status["date"].max().normalize()
    day_status = df_status[df_status["date"] == latest_day].sort_values("status")
    st.subheader("HTTP Status Distribution (latest day)")
    st.dataframe(day_status[["date","status","count"]])
    st.bar_chart(day_status.set_index("status")[["count"]])
else:
    st.info("No status data yet.")

# --- Top URLs (latest day) ---
if not df_top.empty:
    latest_day = df_top["date"].max().normalize()
    day_top = df_top[df_top["date"] == latest_day].sort_values("hits", ascending=False).head(20)
    st.subheader("Top URLs (latest day)")
    st.dataframe(day_top[["date","url","hits"]])
    st.bar_chart(day_top.set_index("url")[["hits"]])
else:
    st.info("No top URL data yet.")

# Auto refresh
if auto:
    st.caption("Auto-refreshing every 5s…")
    time.sleep(5)
    st.rerun()
```

## Description:

This part of the code defines the per-batch aggregations:

1. hits_by_hour: total requests and error counts grouped by date and hour.
2. top_urls: top 1000 requested URLs per day.
3. status_counts: distribution of HTTP response codes.

   Each aggregated DataFrame is written into separate MongoDB collections for dashboard visualization.

## Screenshot 8 — start_all.sh (Bootstrap Scripting for starting)

```
cd ~/spark-hdfs-mongo

bash start_all.sh
```

```
. .venv/bin/activate
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ start_all.sh
start_all.sh: command not found
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ ./start_all.sh
=== STARTING BIG DATA PIPELINE ===
→ Starting HDFS...
→ Starting YARN...
→ Verifying Hadoop processes...
1187 SecondaryNameNode
757 NameNode
904 DataNode
1433 ResourceManager
1580 NodeManager
→ Starting Kafka (KRaft config)…
→ Locating spark-submit...
  Using: /home/deepak/spark-hdfs-mongo/.venv/bin/spark-submit
→ Launching Spark streaming job (Kafka → Mongo)…
  Spark job started (log: /home/deepak/spark-hdfs-mongo/logs/spark_stream.log)
→ Starting Kafka log producer (live timestamps)…
→ Starting Streamlit dashboards…
=== PIPELINE READY ===
✅ Spark streaming job → MongoDB
✅ Kafka producer active
✅ Dashboards:
   → http://localhost:8501 (Alerts)
   → http://localhost:8502 (Analytics)
Logs → /home/deepak/spark-hdfs-mongo/logs
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$
```

```
  GNU nano 7.2                                    start_all.sh
#!/bin/bash
set -euo pipefail

PROJECT_DIR="$HOME/spark-hdfs-mongo"
LOGDIR="$PROJECT_DIR/logs"
PIDDIR="$PROJECT_DIR/pids"
TOPIC="web_logs"
BROKER="localhost:9092"
KAFKA_HOME="$HOME/kafka"

mkdir -p "$LOGDIR" "$PIDDIR"

echo "=== STARTING BIG DATA PIPELINE ==="

# ---------- 0. Python env ----------
if [ -z "${VIRTUAL_ENV:-}" ]; then
  . "$PROJECT_DIR/.venv/bin/activate"
fi

# ---------- 1. HDFS / YARN ----------
echo "→ Starting HDFS..."
start-dfs.sh >/dev/null 2>&1 || true
sleep 2
echo "→ Starting YARN..."
start-yarn.sh >/dev/null 2>&1 || true
sleep 2
```

```
GNU nano 7.2                                    start_all.sh
echo "→ Verifying Hadoop processes..."
jps | grep -E 'NameNode|DataNode|SecondaryNameNode|ResourceManager|NodeManager' || {
  echo "ERROR: Hadoop/YARN not up"; exit 1; }

# ---------- 2. Kafka (KRaft) ----------
echo "→ Starting Kafka (KRaft config)…"
# ensure our kraft config exists (from your earlier setup)
KRAFT_CONF="$KAFKA_HOME/config/kraft/server.properties"
if ! grep -q "process.roles=" "$KRAFT_CONF"; then
  echo "ERROR: Missing KRaft config at $KRAFT_CONF"; exit 1;
fi

# Kill any old broker, then start clean
pkill -f 'kafka\.Kafka' 2>/dev/null || true
nohup "$KAFKA_HOME/bin/kafka-server-start.sh" "$KRAFT_CONF" \
  > /tmp/kafka.log 2>&1 & echo $! > "$PIDDIR/kafka.pid"
sleep 6

# Ensure 9092 is listening
if ! ss -ltn 2>/dev/null | grep -q ':9092'; then
  echo "ERROR: Kafka not listening on 9092; see /tmp/kafka.log"; exit 1;
fi

# Ensure topic
"$KAFKA_HOME/bin/kafka-topics.sh" --bootstrap-server "$BROKER" \
  --create --if-not-exists --topic "$TOPIC" --partitions 1 --replication-factor 1 >/dev/null 2>&1 || true
```

```
GNU nano 7.2                                    start_all.sh

# ---------- 3. Spark streaming job ----------
echo "→ Locating spark-submit..."
if command -v spark-submit >/dev/null 2>&1; then
  SPARK_BIN="$(command -v spark-submit)"
else
  for d in /usr/local/spark /opt/spark "$HOME"/spark-*; do
    [ -x "$d/bin/spark-submit" ] && SPARK_BIN="$d/bin/spark-submit" && break
  done
fi
[ -z "${SPARK_BIN:-}" ] && { echo "ERROR: spark-submit not found"; exit 1; }
echo "  Using: $SPARK_BIN"

echo "→ Launching Spark streaming job (Kafka → Mongo)…"
nohup "$SPARK_BIN" \
  --master yarn \
  --deploy-mode client \
  --packages org.mongodb.spark:mongo-spark-connector_2.12:3.0.2,org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.2 \
  "$PROJECT_DIR/src/stream_logs_kafka.py" \
  > "$LOGDIR/spark_stream.log" 2>&1 & echo $! > "$PIDDIR/spark.pid"
sleep 5
echo "  Spark job started (log: $LOGDIR/spark_stream.log)"

# ---------- 4. Producer (LIVE timestamps) ----------
echo "→ Starting Kafka log producer (live timestamps)…"
nohup "$PROJECT_DIR/produce_kafka.sh" > "$LOGDIR/producer.log" 2>&1 & echo $! > "$PIDDIR/producer.pid"
sleep 2
```

## What it shows:

1. The pipeline bootstrap script with strict bash flags (set -euo pipefail), project paths, and Kafka/topic variables. It creates logs/ and pids/, optionally activates the Python venv, then starts HDFS and YARN (start-dfs.sh, start-yarn.sh) with short waits. Why it matters: Proves your environment is reproducible and that the data plane (HDFS) and resource manager (YARN) are brought up in a controlled order before streaming services.
2. Safety checks that Hadoop daemons are alive (jps), validation that the KRaft config exists, forced cleanup of any stale Kafka broker, then nohup start of the broker with logging to /tmp/kafka.log. It verifies the broker is listening on 9092 and (optionally) ensures the

topic exists.
Why it matters: Demonstrates a clean, scripted Kafka startup with idempotent checks, preventing port conflicts and stale processes.

3. Discovery of the real spark-submit binary, then a nohup launch of your Spark Structured Streaming job (Kafka → MongoDB) in YARN client mode, including required packages (spark-sql-kafka and Mongo connector). Finally, the script kicks off produce_kafka.sh to replay logs with live timestamps.
Why it matters: Evidence that the compute layer (Spark on YARN) is programmatically started, logged (logs/spark_stream.log), and fed by a consistent Kafka producer.

## Screenshot 8 — start_all.sh (Bootstrap Scripting for stopping)

start_all.sh

```
  GNU nano 7.2                                          stop_all.sh
#!/bin/bash
set -euo pipefail
echo "=== STOPPING PIPELINE ==="

# 1) Stop app processes by name (Spark driver, Streamlit, producer)
pkill -f "spark-submit .*stream_logs_kafka.py" 2>/dev/null || true
pkill -f "streamlit run app/alerts.py"         2>/dev/null || true
pkill -f "streamlit run app/analytics.py"      2>/dev/null || true
pkill -f "produce_kafka.sh"                    2>/dev/null || true
pkill -f "python .*streamlit"                  2>/dev/null || true
pkill -f "streamlit"                           2>/dev/null || true

# 2) Free UI ports 8501/8502 aggressively
for p in 8501 8502; do
  fuser -k ${p}/tcp 2>/dev/null || true
  lsof -ti :$p 2>/dev/null | xargs -r kill || true
  sleep 1
  lsof -ti :$p 2>/dev/null | xargs -r kill -9 || true
done

# 3) Stop YARN/HDFS if running (safe if not)
if jps | grep -q ResourceManager; then stop-yarn.sh || true; fi
if jps | grep -Eq 'NameNode|DataNode|SecondaryNameNode'; then stop-dfs.sh || true; fi

# 4) Clean Hadoop PID files
rm -f /tmp/hadoop-*-namenode.pid \
      /tmp/hadoop-*-datanode.pid \
```

```
  GNU nano 7.2                                                    stop_all.sh
pkill -f "streamlit run app/alerts.py"        2>/dev/null || true
pkill -f "streamlit run app/analytics.py"     2>/dev/null || true
pkill -f "produce_kafka.sh"                   2>/dev/null || true
pkill -f "python .*streamlit"                 2>/dev/null || true
pkill -f "streamlit"                          2>/dev/null || true

# 2) Free UI ports 8501/8502 aggressively
for p in 8501 8502; do
  fuser -k ${p}/tcp 2>/dev/null || true
  lsof -ti :$p 2>/dev/null | xargs -r kill || true
  sleep 1
  lsof -ti :$p 2>/dev/null | xargs -r kill -9 || true
done

# 3) Stop YARN/HDFS if running (safe if not)
if jps | grep -q ResourceManager; then stop-yarn.sh || true; fi
if jps | grep -Eq 'NameNode|DataNode|SecondaryNameNode'; then stop-dfs.sh || true; fi

# 4) Clean Hadoop PID files
rm -f /tmp/hadoop-*-namenode.pid \
      /tmp/hadoop-*-datanode.pid \
      /tmp/hadoop-*-secondarynamenode.pid \
      /tmp/hadoop-*-resourcemanager.pid \
      /tmp/hadoop-*-nodemanager.pid 2>/dev/null || true

echo "✅ All components stopped."
```

1. **What it shows:** Controlled teardown: kills Spark driver (spark-submit … stream_logs_kafka.py), Streamlit apps (alerts.py, analytics.py), and the Kafka producer. It aggressively frees UI ports 8501/8502 using fuser/lsof.
   Why it matters: Confirms you have a safe, repeatable shutdown procedure that avoids orphaned processes and port conflicts.
2. Conditional stops for YARN and HDFS (only if running), followed by cleanup of Hadoop PID files for NameNode/DataNode/ResourceManager/NodeManager. Ends with a success confirmation.
   Why it matters: Guarantees a clean cluster state for the next run—critical for grading and reproducibility.

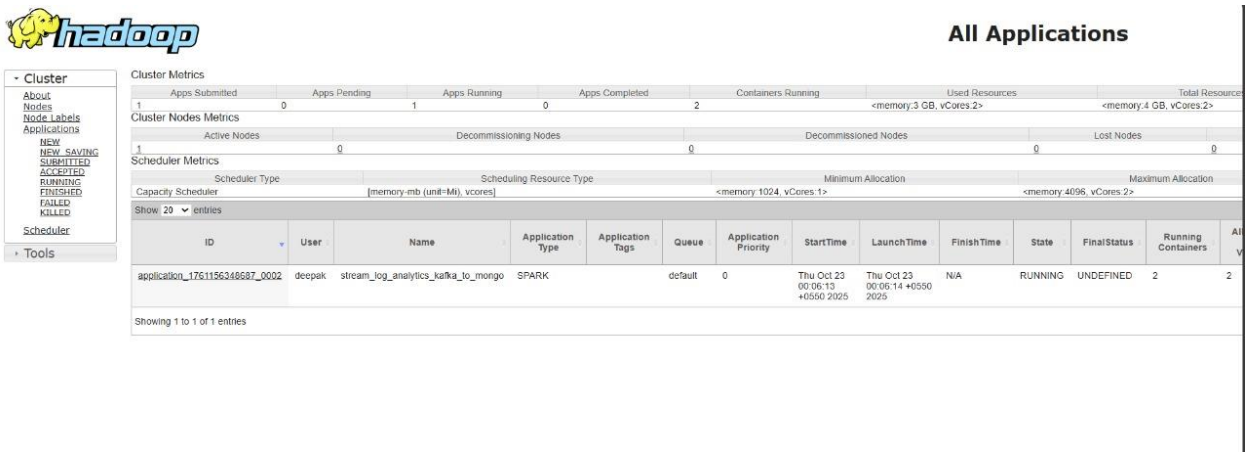
## Commands Table:

| No | Command | Description |
|---|---|---|
| 1 | sudo apt update && sudo apt upgrade -y | Updates and upgrades Ubuntu system packages. |
| 2 | sudo apt install openjdk-11-jdk python3-venv python3-pip -y | Installs Java (for Hadoop/Spark) and Python environment tools. |

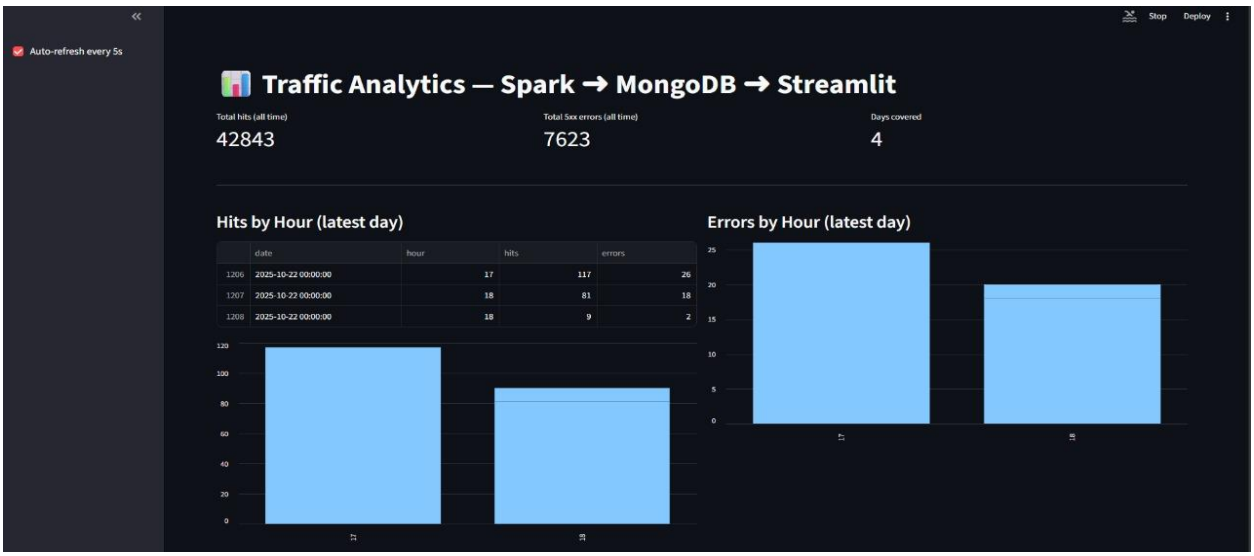
| | | |
|---|---|---|
| 3 | hdfs namenode -format | Formats the Hadoop filesystem before first use. |
| 4 | start-dfs.sh && start-yarn.sh | Starts the HDFS daemons (NameNode/DataNode) and YARN (Resource/Node Managers). |
| 5 | jps | Lists all running Java daemons — verifies Hadoop, YARN, Kafka, etc. |
| 6 | hdfs dfs -mkdir -p /user/deepak | Creates a working directory inside HDFS. |
| 7 | hdfs dfs -put sample_logs/access.log /data/logs/ | Uploads sample logs to HDFS for backup. |
| 8 | hdfs dfs -ls /data/logs | Lists uploaded files inside the HDFS path. |
| 9 | $KAFKA_HOME/bin/kafka-storage.sh format -t $(uuidgen) -c $KAFKA_HOME/config/kraft/server.properties | Initializes Kafka's KRaft metadata directory. |
| 10 | $KAFKA_HOME/bin/kafka-server-start.sh -daemon $KAFKA_HOME/config/kraft/server.properties | Starts the Kafka broker in background mode. |
| 11 | $KAFKA_HOME/bin/kafka-topics.sh --create --topic web_logs --bootstrap-server localhost:9092 | Creates Kafka topic web_logs for streaming log data. |
| 12 | $KAFKA_HOME/bin/kafka-topics.sh --list --bootstrap-server localhost:9092 | Confirms topic creation and lists all topics. |
| 13 | bash produce_kafka.sh | Streams sample Apache logs into Kafka with live timestamps. |
| 14 | spark-submit --master local[2] src/stream_kafka_to_mongo.py | Runs the Spark Structured Streaming job (Kafka → MongoDB) locally. |
| 15 | spark-submit --master yarn --deploy-mode client src/stream_logs_kafka.py | Launches the full analytics job on YARN with MongoDB connector. |
| 16 | mongosh | Opens MongoDB shell for database inspection. |
| 17 | use bdproj | Switches to the project's MongoDB database bdproj. |
| 18 | show collections | Lists MongoDB collections (agg_results, hits_by_hour, etc.). |
| 19 | db.agg_results.find().limit(5) | Displays first 5 aggregated token documents. |
| 20 | db.hits_by_hour.find().limit(5) | Displays hourly hit and error analytics. |
| 21 | bash start_all.sh | One-click startup script for HDFS + YARN + Kafka + Spark + Producer. |

| | | |
|---|---|---|
| 22 | bash stop_all.sh | Gracefully stops all running components and cleans up PID files. |
| 23 | lsof -i :8501 | Checks if Streamlit dashboard ports are free or in use. |
| 24 | streamlit run app/analytics.py | Launches the analytics dashboard to visualize log metrics. |
| 25 | streamlit run app/alerts.py | Starts the alert monitoring dashboard for 5xx error tracking. |
| 26 | hdfs dfs -ls /checkpoints/kafka_logs | Confirms Spark streaming checkpoints are saved in HDFS. |
| 27 | tail -f logs/spark_stream.log | Monitors Spark streaming logs live. |

## OUTPUT PROOF:



Auto-refresh every 5s
Live 5xx Alerts
Spark Structured Streaming → MongoDB → Streamlit
Total alerts (shown)
200
Alerts in last 10 min
1
Last alert (UTC)
2025-10-22 18:00:56
Errors by hour
1,000
800
600
400
200
0
2025-10-18 21
2025-10-18 22
2025-10-17 10
2025-10-17 11
Most recent alerts
created_at date hour errors hits epoch_id
0 2025-10-22 18:00:56 2025-10-17 00:00:00 11 18 81 1029
1 2025-10-17 11:08:29 2025-10-17 00:00:00 11 18 81 1029
2 2025-10-17 11:08:02 2025-10-17 00:00:00 11 18 81 1028
3 2025-10-17 11:07:35 2025-10-17 00:00:00 11 18 81 1027
4 2025-10-17 11:07:03 2025-10-17 00:00:00 11 18 81 1026
5 2025-10-17 11:06:33 2025-10-17 00:00:00 11 20 90 1025
6 2025-10-17 11:06:04 2025-10-17 00:00:00 11 18 81 1024


Stop
Deploy
Auto-refresh every 5s
Traffic Analytics — Spark → MongoDB → Streamlit
Total hits (all time)
42843
Total 5xx errors (all time)
7623
Days covered
4
Hits by Hour (latest day)
date hour hits errors
1206 2025-10-22 00:00:00 17 117 26
1207 2025-10-22 00:00:00 18 81 18
1208 2025-10-22 00:00:00 18 9 2
120
100
80
60
40
20
0
17
18
Errors by Hour (latest day)
25
20
15
10
5
0
17
18

## HTTP Status Distribution (latest day)

| | date | status | count |
|---|---|---|---|
| 4398 | 2025-10-22 00:00:00 | 201 | 22 |
| 4404 | 2025-10-22 00:00:00 | 404 | 1 |
| 4399 | 2025-10-22 00:00:00 | 404 | 22 |
| 4409 | 2025-10-22 00:00:00 | 404 | 2 |
| 4400 | 2025-10-22 00:00:00 | 500 | 22 |
| 4410 | 2025-10-22 00:00:00 | 500 | 2 |
| 4405 | 2025-10-22 00:00:00 | 500 | 1 |
| 4401 | 2025-10-22 00:00:00 | 503 | 22 |
| 4406 | 2025-10-22 00:00:00 | 503 | 1 |
| 4411 | 2025-10-22 00:00:00 | 503 | 2 |

```
+---------+-------------------+------+----------------------+------+-----+-----------+
|127.0.0.1|2025-10-22 18:02:00|GET   |/api/v1/users HTTP/1.1 |200   |512  |Mozilla/5.0|
|127.0.0.1|2025-10-22 18:02:00|GET   |/api/v1/orders HTTP/1.1|200   |1024 |Mozilla/5.0|
|127.0.0.1|2025-10-22 18:02:00|POST  |/api/v1/orders HTTP/1.1|201   |256  |curl/8.0   |
|127.0.0.1|2025-10-22 18:02:00|GET   |/health HTTP/1.1       |200   |64   |curl/8.0   |
|127.0.0.1|2025-10-22 18:02:00|GET   |/api/v1/users HTTP/1.1 |500   |0    |Mozilla/5.0|
+---------+-------------------+------+----------------------+------+-----+-----------+
only showing top 5 rows

ALERTS:
+----------+----+----+------+--------+-------------------------+
|date      |hour|hits|errors|epoch_id|created_at               |
+----------+----+----+------+--------+-------------------------+
|2025-10-22|18  |18  |4     |1035    |2025-10-22 18:02:21.070107|
+----------+----+----+------+--------+-------------------------+

=== Batch 1036 sample ===
+---------+-------------------+------+----------------------+------+-----+-----------+
|ip       |ts                 |method|url                   |status|bytes|agent      |
+---------+-------------------+------+----------------------+------+-----+-----------+
|127.0.0.1|2025-10-22 18:02:15|GET   |/api/v1/users HTTP/1.1 |200   |512  |Mozilla/5.0|
|127.0.0.1|2025-10-22 18:02:15|GET   |/api/v1/orders HTTP/1.1|200   |1024 |Mozilla/5.0|
|127.0.0.1|2025-10-22 18:02:15|POST  |/api/v1/orders HTTP/1.1|201   |256  |curl/8.0   |
|127.0.0.1|2025-10-22 18:02:15|GET   |/health HTTP/1.1       |200   |64   |curl/8.0   |
|127.0.0.1|2025-10-22 18:02:15|GET   |/api/v1/users HTTP/1.1 |500   |0    |Mozilla/5.0|
+---------+-------------------+------+----------------------+------+-----+-----------+
only showing top 5 rows
```

```
. .venv/bin/activate
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ start_all.sh
start_all.sh: command not found
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ ./start_all.sh
=== STARTING BIG DATA PIPELINE ===
→ Starting HDFS...
→ Starting YARN...
→ Verifying Hadoop processes...
1187 SecondaryNameNode
757 NameNode
904 DataNode
1433 ResourceManager
1580 NodeManager
→ Starting Kafka (KRaft config)…
→ Locating spark-submit...
  Using: /home/deepak/spark-hdfs-mongo/.venv/bin/spark-submit
→ Launching Spark streaming job (Kafka → Mongo)…
  Spark job started (log: /home/deepak/spark-hdfs-mongo/logs/spark_stream.log)
→ Starting Kafka log producer (live timestamps)…
→ Starting Streamlit dashboards…
=== PIPELINE READY ===
✅ Spark streaming job → MongoDB
✅ Kafka producer active
✅ Dashboards:
   → http://localhost:8501 (Alerts)
   → http://localhost:8502 (Analytics)
Logs → /home/deepak/spark-hdfs-mongo/logs
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$
```